

MOMO Quant

Part 2 — System Architecture Document

Version: 1.0 Draft

Architecture Style: Modular Monolith + Python AI Service

Primary Goal: Build a maintainable solo-developer trading platform without unnecessary infrastructure complexity.

1. Architecture Decision

MOMO Quant will start as a **modular monolith**, not microservices.

This means the backend will be one .NET application, but internally separated into clean modules.

Why

Microservices are overkill at the start.

They add:

- deployment complexity
- networking issues
- debugging complexity
- distributed logging problems
- more DevOps work
- more failure points

For one developer, that is a bad trade-off.

The correct first architecture is:

```
React Dashboard
  |
  v
.NET 8 Backend API
  |
  +-- Market Data Module
  +-- Indicator Module
  +-- Strategy Module
  +-- AI Decision Module
  +-- Risk Module
  +-- Execution Module
```

```
    +-- Backtesting Module
    +-- Replay Module
    +-- Paper Trading Module
    +-- Reporting Module
    +-- Monitoring Module
    |
    +-- MySQL
    +-- Redis
    |
    v
Python AI Service
```

2. Main Applications

MOMO Quant will contain four main runtime components:

2.1 React Dashboard

Used for:

- bot control
- reports
- strategy monitoring
- replay mode
- settings
- logs
- paper trading view
- future live trading view

2.2 .NET Backend

Main brain of the system.

Responsible for:

- authentication
- APIs
- trading workflow
- market data ingestion
- indicators
- strategies
- risk checks
- execution simulation
- reporting

- monitoring
 - audit logging
-

2.3 Python AI Service

Used for:

- market regime detection
- confidence scoring
- parameter optimization
- anomaly detection
- trade explanation support

The AI service must not directly place trades.

2.4 Database and Cache

Use:

- MySQL for persistent data
 - Redis for cache, runtime state, queues, and fast pub/sub if needed
-

3. Backend Solution Structure

Recommended .NET solution:

```
MomoQuant.sln

src/
  MomoQuant.Api/
  MomoQuant.Application/
  MomoQuant.Domain/
  MomoQuant.Infrastructure/
  MomoQuant.Persistence/
  MomoQuant.Worker/
  MomoQuant.Shared/

tests/
  MomoQuant.UnitTests/
  MomoQuant.IntegrationTests/
```

4. Project Responsibilities

4.1 MomoQuant.Api

Contains:

- controllers
- SignalR hubs
- authentication endpoints
- dashboard APIs
- request/response DTOs
- middleware
- API configuration

Should not contain trading logic.

4.2 MomoQuant.Application

Contains business use cases:

- run backtest
- start paper trading
- stop bot
- switch mode
- calculate report
- submit strategy signal
- evaluate trade decision

This layer coordinates modules.

4.3 MomoQuant.Domain

Contains core domain logic:

- Strategy interfaces
- Trade models
- Order models
- Risk models
- Position models
- Signal models
- Market regime models
- Enums
- Domain rules

This layer should not depend on database, API, or exchange libraries.

4.4 MomoQuant.Infrastructure

Contains external integrations:

- exchange clients
- WebSocket clients
- REST clients
- Redis services
- notification providers
- Python AI HTTP client
- logging integrations

4.5 MomoQuant.Persistence

Contains:

- Entity Framework DbContext
- database entities
- migrations
- repositories
- unit of work
- query services

4.6 MomoQuant.Worker

Background processes:

- market data collector
- candle builder
- paper trading loop
- bot engine loop
- scheduled reporting
- health checks
- order reconciliation

4.7 MomoQuant.Shared

Reusable shared contracts:

- constants
- shared DTOs
- result models

- error models
- pagination models

Keep this small.

5. Backend Module Structure

Inside `MomoQuant.Application`, use module folders:

```
Application/  
  Auth/  
  Users/  
  Exchanges/  
  MarketData/  
  Indicators/  
  Strategies/  
  AI/  
  Risk/  
  Execution/  
  Backtesting/  
  Replay/  
  PaperTrading/  
  LiveTrading/  
  Reporting/  
  Monitoring/  
  AuditLogs/  
  Settings/
```

Each module should contain:

```
Commands/  
Queries/  
Services/  
DTOs/  
Validators/  
Mappings/
```

6. Domain Module Structure

```
Domain/  
  Common/
```

```
MarketData/  
Indicators/  
Strategies/  
AI/  
Risk/  
Execution/  
Backtesting/  
Trading/  
Reporting/  
Monitoring/
```

7. Strategy Architecture

Every strategy must implement:

```
public interface ITradingStrategy  
{  
    string Name { get; }  
  
    StrategySignal Evaluate(StrategyContext context);  
  
    IReadOnlyCollection<MarketRegime> SupportedRegimes { get; }  
  
    IReadOnlyCollection<Timeframe> SupportedTimeframes { get; }  
}
```

Initial strategies:

```
Strategies/  
    LiquiditySweepStrategy  
    VwapMeanReversionStrategy  
    EmaPullbackStrategy
```

Strategies must not place orders.

They only produce signals.

8. Trading Decision Flow

Correct flow:

```
Market Data
|
v
Indicator Engine
|
v
Market Regime Detector
|
v
Strategy Engine
|
v
AI Confidence Engine
|
v
Risk Engine
|
v
Execution Engine
|
v
Trade Logger
```

Important rule:

Execution happens only after risk approval.

9. Trading Modes

All modes must share the same core logic.

```
Backtest Execution Provider
Paper Execution Provider
Live Execution Provider
```

They should implement the same interface:

```
public interface IExecutionProvider
{
    Task<OrderResult> PlaceOrderAsync(OrderRequest request, CancellationToken
cancellationToken);
    Task<OrderStatus> GetOrderStatusAsync(string orderId, CancellationToken
cancellationToken);
```



```
Task<CancelOrderResult> CancelOrderAsync(string orderId, CancellationToken
cancellationToken);
}
```

This prevents duplicated logic.

10. Mode Safety

Trading mode must be controlled centrally.

```
Backtest
Replay
Paper
Live
```

Live mode must require:

- admin role
- explicit confirmation
- exchange credentials configured
- risk rules enabled
- emergency stop available
- paper trading validation completed

Default mode: **Paper** or **Backtest**, never Live.

11. AI Service Architecture

Python AI service exposes HTTP endpoints:

```
POST /ai/regime/detect
POST /ai/confidence/score
POST /ai/parameters/optimize
POST /ai/anomaly/detect
POST /ai/trade/explain
```

The .NET backend calls the AI service.

The AI service returns structured JSON.

Example:

```
{
  "marketRegime": "Trending",
  "confidence": 86,
  "riskAdjustment": "Normal",
  "preferredStrategy": "EMA_PULLBACK",
  "tradeAllowed": true,
  "explanation": "Trend is bullish, EMA alignment is valid, volatility is acceptable."
}
```

AI output is advisory.

Risk engine can still reject the trade.

12. Risk Engine Architecture

Risk engine must be independent.

It receives:

- signal
- AI decision
- account state
- open positions
- symbol volatility
- spread
- daily PnL
- risk settings

It returns:

```
Approved
Rejected
Rejected with reason
Adjusted position size
Adjusted stop loss
Adjusted take profit
```

The risk engine is the final gate before execution.

13. Execution Architecture

Execution module must support:

- backtest fills
- paper fills
- live exchange orders later

Order lifecycle:

```
Created
Submitted
PartiallyFilled
Filled
Cancelled
Rejected
Expired
Failed
```

Maker limit order logic must include:

- post-only flag
- order timeout
- cancel if not filled
- partial fill tracking
- missed trade logging

14. Market Data Architecture

Market data comes from:

- exchange WebSocket
- exchange REST historical API
- imported historical files if needed

Market data types:

- candles
- trades
- order book
- funding rates
- open interest

Initial MVP should start with candles first.

Do not overbuild order book logic in v1.

15. Candle Storage

Store normalized candles:

- symbol
- exchange
- timeframe
- open time
- close time
- open
- high
- low
- close
- volume

Indexes required:

- symbol + timeframe + open time
 - exchange + symbol
 - open time
-

16. Indicator Engine

Indicator engine calculates:

- EMA
- VWAP
- RSI
- ATR
- Volume SMA
- Swing highs/lows
- Market structure

Indicators should be deterministic and testable.

Do not calculate indicators directly inside strategies.

17. Backtesting Architecture

Backtesting uses historical candles and simulates market progress candle by candle.

Flow:

```
Load historical candles
|
v
Replay candles internally
|
v
Calculate indicators
|
v
Run strategy engine
|
v
Run AI/rules confidence
|
v
Run risk engine
|
v
Simulate execution
|
v
Calculate performance
```

Backtesting must include:

- fees
- spread
- slippage assumptions
- maker fill assumptions
- missed trades
- partial fills later

18. Replay Mode Architecture

Replay mode uses the backtesting engine but exposes the timeline to the UI.

It must show:

- candle movement
- indicators
- signals
- AI decisions

- risk decisions
- execution decisions
- trade results
- no-trade reasons

Replay mode is not optional. It is essential for debugging.

19. Paper Trading Architecture

Paper trading uses live market data with simulated execution.

Flow:

```
Live WebSocket data
|
v
Indicator Engine
|
v
Strategy Engine
|
v
AI Confidence Engine
|
v
Risk Engine
|
v
Paper Execution Provider
|
v
Paper Portfolio
```

Paper trading must behave as close to live trading as possible.

20. Reporting Architecture

Reports are generated from stored trades, signals, orders, and decisions.

Reporting service must calculate:

- PnL
- win rate

- drawdown
 - profit factor
 - expectancy
 - fees
 - strategy performance
 - symbol performance
 - regime performance
 - rejection reasons
 - missed trade reasons
-

21. Monitoring Architecture

Monitoring should track:

- backend status
- worker status
- AI service status
- database status
- Redis status
- exchange connection status
- WebSocket reconnect count
- API latency
- order latency
- bot current mode
- emergency stop status

Dashboard should show:

```
Safe
Warning
Critical
Stopped
```

22. Real-Time Updates

Use SignalR for dashboard real-time updates.

SignalR channels:

```
/bot/status
/market/live
/trades/live
```

```
/orders/live
/monitoring/live
/replay/live
```

Events:

- BotStatusChanged
- SignalGenerated
- TradeOpened
- TradeClosed
- OrderUpdated
- RiskRejected
- AiDecisionCreated
- HealthChanged

23. Configuration Architecture

Settings should be stored in both:

- appsettings.json for system defaults
- database for runtime configurable values

Examples:

- enabled symbols
- strategy parameters
- risk limits
- trading mode
- exchange settings
- AI settings
- report settings

Secrets must not be stored as plain text.

24. Suggested Deployment

Initial deployment:

```
Docker Compose
```

```
containers:
  momo-api
  momo-ai
```



```
momo-mysql  
momo-redis  
momo-dashboard
```

Avoid Kubernetes in v1.

25. Cursor Implementation Rule

Cursor must build this project module by module.

Do not ask Cursor to generate the whole platform at once.

Correct build order:

1. Solution skeleton
2. Domain models
3. Database schema
4. Market data module
5. Indicator module
6. Strategy module
7. Backtesting module
8. Replay module
9. Paper trading module
10. Reporting module
11. Monitoring module
12. AI service integration
13. Dashboard
14. Live trading later

26. Critical Architecture Rules

The system must follow these rules:

1. Strategies never place orders.
2. AI never places orders.
3. Risk engine has final authority.
4. Backtest, paper, and live modes share core logic.
5. Execution provider changes by mode.
6. Every decision is logged.
7. Live trading is disabled by default.
8. Modules must be testable independently.
9. Database logic must not leak into domain logic.

10. Dashboard must not contain trading decisions.
 11. No hidden magic logic.
 12. Every trade must be explainable.
-

27. Architecture Summary

MOMO Quant will begin as a clean modular monolith with a separate Python AI service.

This is the right architecture for a solo developer because it avoids premature infrastructure complexity while still keeping the system organized.

The first version should prioritize:

- correctness
- testability
- replayability
- risk safety
- reporting
- explainability

Do not optimize for scale before the system is profitable and stable.